

Computer Architecture

Project-2

Group11

Group Members:

Name	ID
Pritham Kumar Jena	2023A8PS1308H
Maanya Saikia	2022B3A81654H
R Revanth Kanna	2022B4AA0926H
Saumya Priyadarsan Mohapatra	2022B4AA0992H

*All members have equally contributed in the project

Problem Statement:

Design a 5-stage (as discussed in the lecture) pipeline RISC processor that can execute the following instructions. The processor design must also include a hazard detection and stall/forwarding unit.

SUBI reg1, reg2, 15

LW reg4, 0(reg1)

LW reg5, 8(reg1)

NAND reg6, reg8, reg5

SW reg5, 0(reg7)

Initialise the register file with the following data:

reg2 = $(25)_{16}$, reg8 = $(9878C)_{16}$, reg7 = $(C)_{16}$

Verilog Codes

Top-Module:

```
module top (
    input clk,
    input reset
);
    wire stall;
    wire PCWrite    = ~stall;
    wire IFIDWrite  = ~stall;
    wire [31:0] pc_out;
    wire [31:0] pc_plus4 = pc_out + 32'd4;
    PC pc_inst (
        .clk      (clk),
        .reset     (reset),
        .PCWrite   (PCWrite),
        .pc_next   (pc_plus4),
        .pc_out    (pc_out)
    );
    wire [31:0] if_instr;
    instruction_memory imem (
        .address   (pc_out),
        .readdata  (if_instr)
    );
    wire [31:0] if_id_pc_plus4;
    wire [31:0] if_id_instr;

    if_id_reg if_id_reg_inst (
        .clk      (clk),
        .reset     (reset),
        .IFIDWrite (IFIDWrite),
        .pc_plus4_in (pc_plus4),
        .instr_in   (if_instr),
        .pc_plus4_out (if_id_pc_plus4),
        .instr_out  (if_id_instr)
    );
    wire [3:0] id_opcode = if_id_instr[31:28];
    wire [3:0] id_rd     = if_id_instr[27:24];
    wire [3:0] id_rs     = if_id_instr[23:20];
    wire [3:0] id_rt     = if_id_instr[19:16];
    wire [15:0] id_imm    = if_id_instr[15:0];
    wire id_regwrite, id_memtoreg, id_memread, id_memwrite, id_alusrc;
    wire [1:0] id_aluop;

    controlunit ctrl (
        .opcode    (id_opcode),
        .RegWrite  (id_regwrite),
        .MemToReg  (id_memtoreg),
        .MemRead   (id_memread),
        .MemWrite  (id_memwrite),
```

```

        .ALUSrc    (id_alusrc),
        .ALUOp     (id_aluop)
    );
    wire [3:0]  wb_rd;
    wire [31:0] wb_writedata;
    wire wb_regwrite;
    wire [31:0] id_readdata1, id_readdata2;

    regbank regbank_inst (
        .clk        (clk),
        .rs          (id_rs),
        .rt          (id_rt),
        .readdata1   (id_readdata1),
        .readdata2   (id_readdata2),
        .rd          (wb_rd),
        .writedata   (wb_writedata),
        .regwrite    (wb_regwrite)
    );
    wire [31:0] id_signext;

    signextend sext (
        .imm_in      (id_imm),
        .imm_out     (id_signext)
    );
    wire [3:0] id_ex_rd;
    wire      id_ex_memread;

    hazard_detection hazard_det (
        .id_ex_memread (id_ex_memread),
        .id_ex_rd      (id_ex_rd),
        .if_id_rs      (id_rs),
        .if_id_rt      (id_rt),
        .stall         (stall)
    );
    wire [31:0] id_ex_pc_plus4;
    wire [31:0] id_ex_readdata1;
    wire [31:0] id_ex_readdata2;
    wire [31:0] id_ex_signext;
    wire [3:0]  id_ex_rs, id_ex_rt;
    wire id_ex_regwrite, id_ex_memtoreg;
    wire id_ex_memwrite, id_ex_alusrc;
    wire [1:0]  id_ex_aluop;

    id_ex_reg id_ex_reg_inst (
        .clk        (clk),
        .reset       (reset),
        .pc_plus4_in (if_id_pc_plus4),
        .readdata1_in (id_readdata1),
        .readdata2_in (id_readdata2),
        .signext_in  (id_signext),
        .rs_in       (id_rs),
        .rt_in       (id_rt),

```

```

        .rd_in          (id_rd),
        .RegWrite_in   (id_regwrite),
        .MemToReg_in   (id_memtoreg),
        .MemRead_in    (id_memread),
        .MemWrite_in   (id_memwrite),
        .ALUSrc_in     (id_alusrc),
        .ALUOp_in      (id_aluop),
        .stall         (stall),
        .pc_plus4_out   (id_ex_pc_plus4),
        .readdata1_out  (id_ex_readdata1),
        .readdata2_out  (id_ex_readdata2),
        .signext_out    (id_ex_signext),
        .rs_out         (id_ex_rs),
        .rt_out         (id_ex_rt),
        .rd_out         (id_ex_rd),
        .RegWrite_out   (id_ex_regwrite),
        .MemToReg_out   (id_ex_memtoreg),
        .MemRead_out    (id_ex_memread),
        .MemWrite_out   (id_ex_memwrite),
        .ALUSrc_out     (id_ex_alusrc),
        .ALUOp_out      (id_ex_aluop)
    );
    wire [3:0] ex_mem_rd;
    wire      ex_mem_regwrite;
    wire [3:0] mem_wb_rd;
    wire      mem_wb_regwrite;
    wire [31:0] ex_mem_alu_result;
    wire      mem_wb_memtoreg;
    wire [31:0] mem_wb_alu_result;
    wire [31:0] mem_wb_mem_readdata;
    assign wb_writedata = mem_wb_memtoreg ? mem_wb_mem_readdata
                                         : mem_wb_alu_result;

    assign wb_rd          = mem_wb_rd;
    assign wb_regwrite    = mem_wb_regwrite;
    wire [1:0] forwardA, forwardB;
    forwarding_unit fwd_unit (
        .id_ex_rs         (id_ex_rs),
        .id_ex_rt         (id_ex_rt),
        .ex_mem_rd        (ex_mem_rd),
        .ex_mem_regwrite   (ex_mem_regwrite),
        .mem_wb_rd         (mem_wb_rd),
        .mem_wb_regwrite   (mem_wb_regwrite),
        .forwardA          (forwardA),
        .forwardB          (forwardB)
    );
    wire [31:0] alu_operandA =
        (forwardA == 2'b10) ? ex_mem_alu_result :
        (forwardA == 2'b01) ? wb_writedata      :
                               id_ex_readdata1;
    wire [31:0] forwardB_out =
        (forwardB == 2'b10) ? ex_mem_alu_result :
        (forwardB == 2'b01) ? wb_writedata      :

```

```

                                id_ex_readdata2;
wire [31:0] alu_operandB = id_ex_alusrc ? id_ex_signext : forwardB_out;
wire [31:0] ex_alu_result;
alu alu_inst (
    .operandA      (alu_operandA),
    .operandB      (alu_operandB),
    .alu_control   (id_ex_aluop),
    .result        (ex_alu_result)
);
wire [31:0] ex_mem_writedata;
wire        ex_mem_memtoreg;
wire        ex_mem_memread;
wire        ex_mem_memwrite;
ex_mem_reg ex_mem_reg_inst (
    .clk          (clk),
    .reset        (reset),
    .alu_result_in (ex_alu_result),
    .writedata_in  (forwardB_out),
    .rd_in         (id_ex_rd),
    .RegWrite_in   (id_ex_regwrite),
    .MemToReg_in   (id_ex_memtoreg),
    .MemRead_in    (id_ex_memread),
    .MemWrite_in   (id_ex_memwrite),
    .alu_result_out (ex_mem_alu_result),
    .writedata_out (ex_mem_writedata),
    .rd_out        (ex_mem_rd),
    .RegWrite_out  (ex_mem_regwrite),
    .MemToReg_out  (ex_mem_memtoreg),
    .MemRead_out   (ex_mem_memread),
    .MemWrite_out  (ex_mem_memwrite)
);
wire [31:0] mem_readdata;
data_memory dmem (
    .clk          (clk),
    .reset        (reset),
    .address      (ex_mem_alu_result),
    .writedata     (ex_mem_writedata),
    .memread      (ex_mem_memread),
    .memwrite     (ex_mem_memwrite),
    .readdata     (mem_readdata)
);
mem_wb_reg mem_wb_reg_inst (
    .clk          (clk),
    .reset        (reset),
    .mem_readdata_in (mem_readdata),
    .alu_result_in  (ex_mem_alu_result),
    .rd_in         (ex_mem_rd),
    .RegWrite_in   (ex_mem_regwrite),
    .MemToReg_in   (ex_mem_memtoreg),
    .mem_readdata_out (mem_wb_mem_readdata),
    .alu_result_out (mem_wb_alu_result),
    .rd_out        (mem_wb_rd),

```

```

        .RegWrite_out      (mem_wb_regwrite),
        .MemToReg_out      (mem_wb_memtoreg)
    );
endmodule

```

Program Counter:

```

module PC (
    input clk,
    input reset,
    input PCWrite,
    input [31:0] pc_next,
    output reg [31:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 32'b0;
        else if (PCWrite)
            pc_out <= pc_next;
    end
endmodule

```

Instruction Memory:

```

module instruction_memory (
    input [31:0] address,
    output [31:0] readdata
);
    reg [31:0] memory [0:63];
    assign readdata = memory[address[7:2]];
endmodule

```

IF-ID Reg:

```

module if_id_reg (
    input clk,
    input reset,
    input IFIDWrite,
    input [31:0] pc_plus4_in,
    input [31:0] instr_in,
    output reg [31:0] pc_plus4_out,
    output reg [31:0] instr_out
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            pc_plus4_out <= 32'b0;
            instr_out <= 32'b0;
        end
    end
endmodule

```

```

        end else if (IFIDWrite) begin
            pc_plus4_out <= pc_plus4_in;
            instr_out <= instr_in;
        end
    end
endmodule

```

Control Unit:

```

module controlunit (
    input [3:0] opcode,
    output reg RegWrite,
    output reg MemToReg,
    output reg MemRead,
    output reg MemWrite,
    output reg ALUSrc,
    output reg [1:0] ALUOp
);
    always @(*) begin
        RegWrite = 0;  MemToReg = 0;
        MemRead  = 0;  MemWrite = 0;
        ALUSrc   = 0;  ALUOp    = 2'b00;

        case (opcode)
            4'b0000: begin
                RegWrite = 1;
                ALUSrc   = 1;
                ALUOp    = 2'b01;
            end
            4'b0100: begin
                RegWrite = 1;
                MemToReg = 1;
                MemRead  = 1;
                ALUSrc   = 1;
                ALUOp    = 2'b00;
            end
            4'b0010: begin
                RegWrite = 1;
                ALUSrc   = 0;
                ALUOp    = 2'b10;
            end
            4'b0001: begin
                MemWrite = 1;
                ALUSrc   = 1;
                ALUOp    = 2'b00;
            end
            default: ;
        endcase
    end
endmodule

```

Regbank:

```
module regbank (  
    input clk,  
    input [3:0] rs,  
    input [3:0] rt,  
    output [31:0] readdata1,  
    output [31:0] readdata2,  
    input [3:0] rd,  
    input [31:0] writedata,  
    input regwrite  
);  
    reg [31:0] regs [0:15];  
    assign readdata1 = (regwrite && (rd == rs)) ? writedata : regs[rs];  
    assign readdata2 = (regwrite && (rd == rt)) ? writedata : regs[rt];  
    always @(posedge clk)  
        if (regwrite)  
            regs[rd] <= writedata;  
endmodule
```

Sign-Extend:

```
module signextend (  
    input [15:0] imm_in,  
    output [31:0] imm_out  
);  
    assign imm_out = {{16{imm_in[15]}}, imm_in};  
endmodule
```

Hazard Detection:

```
module hazard_detection (  
    input id_ex_memread,  
    input [3:0] id_ex_rd,  
    input [3:0] if_id_rs,  
    input [3:0] if_id_rt,  
    output reg stall  
);  
    always @(*) begin  
        stall = id_ex_memread &&  
                ((id_ex_rd == if_id_rs) || (id_ex_rd == if_id_rt));  
    end  
endmodule
```


ID-EX Reg:

```
module id_ex_reg (
    input clk,
    input reset,
    input [31:0] pc_plus4_in,
    input [31:0] readdata1_in,
    input [31:0] readdata2_in,
    input [31:0] signext_in,
    input [3:0] rs_in,
    input [3:0] rt_in,
    input [3:0] rd_in,
    input RegWrite_in,
    input MemToReg_in,
    input MemRead_in,
    input MemWrite_in,
    input ALUSrc_in,
    input [1:0] ALUOp_in,
    input stall,
    output reg [31:0] pc_plus4_out,
    output reg [31:0] readdata1_out,
    output reg [31:0] readdata2_out,
    output reg [31:0] signext_out,
    output reg [3:0] rs_out,
    output reg [3:0] rt_out,
    output reg [3:0] rd_out,
    output reg RegWrite_out,
    output reg MemToReg_out,
    output reg MemRead_out,
    output reg MemWrite_out,
    output reg ALUSrc_out,
    output reg [1:0] ALUOp_out
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            pc_plus4_out    <= 32'b0;
            readdata1_out   <= 32'b0;
            readdata2_out   <= 32'b0;
            signext_out     <= 32'b0;
            rs_out          <= 4'b0;
            rt_out          <= 4'b0;
            rd_out          <= 4'b0;
            RegWrite_out    <= 0;  MemToReg_out <= 0;
            MemRead_out     <= 0;  MemWrite_out <= 0;
            ALUSrc_out      <= 0;  ALUOp_out    <= 2'b0;
        end else begin
            pc_plus4_out    <= pc_plus4_in;
            readdata1_out   <= readdata1_in;
            readdata2_out   <= readdata2_in;
            signext_out     <= signext_in;
            rs_out          <= rs_in;
            rt_out          <= rt_in;
```

```

        rd_out      <= rd_in;
    if (stall) begin
        RegWrite_out <= 0;  MemToReg_out <= 0;
        MemRead_out  <= 0;  MemWrite_out <= 0;
        ALUSrc_out   <= 0;  ALUOp_out    <= 2'b0;
    end else begin
        RegWrite_out <= RegWrite_in;  MemToReg_out <= MemToReg_in;
        MemRead_out  <= MemRead_in;   MemWrite_out <= MemWrite_in;
        ALUSrc_out   <= ALUSrc_in;    ALUOp_out    <= ALUOp_in;
    end
end
end
endmodule

```

Forwarding Unit:

```

module forwarding_unit (
    input [3:0] id_ex_rs,
    input [3:0] id_ex_rt,
    input [3:0] ex_mem_rd,
    input ex_mem_regwrite,
    input [3:0] mem_wb_rd,
    input mem_wb_regwrite,
    output reg [1:0] forwardA,
    output reg [1:0] forwardB
);
    always @(*) begin
        if (ex_mem_regwrite && (ex_mem_rd != 4'b0) &&
            (ex_mem_rd == id_ex_rs))
            forwardA = 2'b10;
        else if (mem_wb_regwrite && (mem_wb_rd != 4'b0) &&
            (mem_wb_rd == id_ex_rs))
            forwardA = 2'b01;
        else
            forwardA = 2'b00;
        if (ex_mem_regwrite && (ex_mem_rd != 4'b0) &&
            (ex_mem_rd == id_ex_rt))
            forwardB = 2'b10;
        else if (mem_wb_regwrite && (mem_wb_rd != 4'b0) &&
            (mem_wb_rd == id_ex_rt))
            forwardB = 2'b01;
        else
            forwardB = 2'b00;
    end
endmodule

```

ALU:

```
module alu (  
    input [31:0] operandA,  
    input [31:0] operandB,  
    input [1:0] alu_control,  
    output reg [31:0] result  
);  
    always @(*) begin  
        case (alu_control)  
            2'b00: result = operandA + operandB;  
            2'b01: result = operandA - operandB;  
            2'b10: result = ~(operandA & operandB);  
            default: result = 32'b0;  
        endcase  
    end  
endmodule
```

EX-MEM Reg:

```
module ex_mem_reg (  
    input clk,  
    input reset,  
    input [31:0] alu_result_in,  
    input [31:0] writedata_in,  
    input [3:0] rd_in,  
    input RegWrite_in,  
    input MemToReg_in,  
    input MemRead_in,  
    input MemWrite_in,  
    output reg [31:0] alu_result_out,  
    output reg [31:0] writedata_out,  
    output reg [3:0] rd_out,  
    output reg RegWrite_out,  
    output reg MemToReg_out,  
    output reg MemRead_out,  
    output reg MemWrite_out  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            alu_result_out <= 32'b0;  
            writedata_out <= 32'b0;  
            rd_out <= 4'b0;  
            RegWrite_out <= 0; MemToReg_out <= 0;  
            MemRead_out <= 0; MemWrite_out <= 0;  
        end else begin  
            alu_result_out <= alu_result_in;  
            writedata_out <= writedata_in;  
            rd_out <= rd_in;  
            RegWrite_out <= RegWrite_in;
```

```

        MemToReg_out    <= MemToReg_in;
        MemRead_out     <= MemRead_in;
        MemWrite_out    <= MemWrite_in;
    end
end
endmodule

```

Data Memory:

```

module data_memory (
    input      clk,
    input      reset,
    input [31:0] address,
    input [31:0] writedata,
    input      memread,
    input      memwrite,
    output [31:0] readdata
);
    reg [31:0] memory [0:63];
    assign readdata = memread ? memory[address[7:2]] : 32'b0;
    always @(posedge clk) begin
        if (memwrite)
            memory[address[7:2]] <= writedata;
    end
endmodule

```

MEM-WB Reg:

```

module mem_wb_reg (
    input clk,
    input reset,
    input [31:0] mem_readdata_in,
    input [31:0] alu_result_in,
    input [3:0] rd_in,
    input RegWrite_in,
    input MemToReg_in,
    output reg [31:0] mem_readdata_out,
    output reg [31:0] alu_result_out,
    output reg [3:0] rd_out,
    output reg RegWrite_out,
    output reg MemToReg_out
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            mem_readdata_out <= 32'b0;
            alu_result_out   <= 32'b0;
            rd_out           <= 4'b0;
            RegWrite_out     <= 0;
            MemToReg_out     <= 0;
        end
    end
endmodule

```

```

        end else begin
            mem_readdata_out <= mem_readdata_in;
            alu_result_out   <= alu_result_in;
            rd_out           <= rd_in;
            RegWrite_out     <= RegWrite_in;
            MemToReg_out     <= MemToReg_in;
        end
    end
endmodule

```

Testbench Code

```

`timescale 1ns/1ps
module testbench;
    reg clk, reset;

    top dut (
        .clk   (clk),
        .reset (reset)
    );
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin

        reset = 1;
        repeat(2) @(posedge clk);
        @(negedge clk);
        reset = 0;
        dut.imem.memory[0] = 32'h0120_000F;
        dut.imem.memory[1] = 32'h4410_0000;
        dut.imem.memory[2] = 32'h4510_0008;
        dut.imem.memory[3] = 32'h2685_0000;
        dut.imem.memory[4] = 32'h1075_0000;
        dut.regbank_inst.regs[2] = 32'h0000_0025;
        dut.regbank_inst.regs[7] = 32'h0000_000C;
        dut.regbank_inst.regs[8] = 32'h0009_878C;
        dut.dmem.memory[5] = 32'hAABB_CCDD;
        dut.dmem.memory[7] = 32'h1122_3344;
        #100
        $finish;
    end
    initial begin
        $dumpfile("pipeline_sim.vcd");
        $dumpvars(0, testbench);
    end
endmodule

```

Pipeline Register Sizes

IF/ID Register : 64 bits

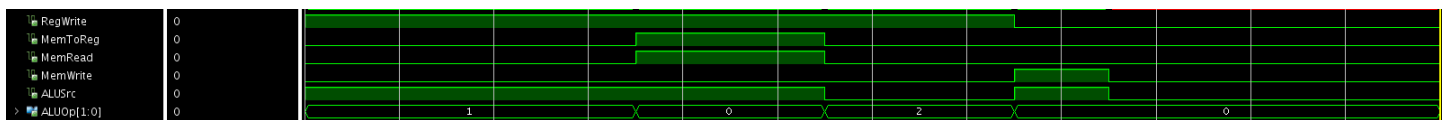
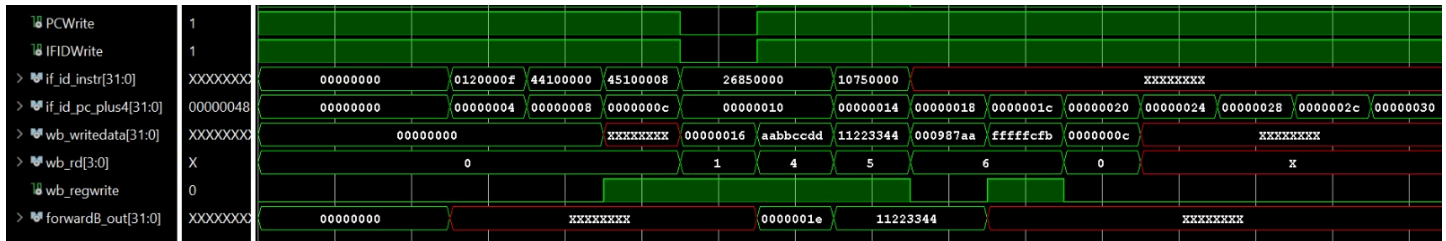
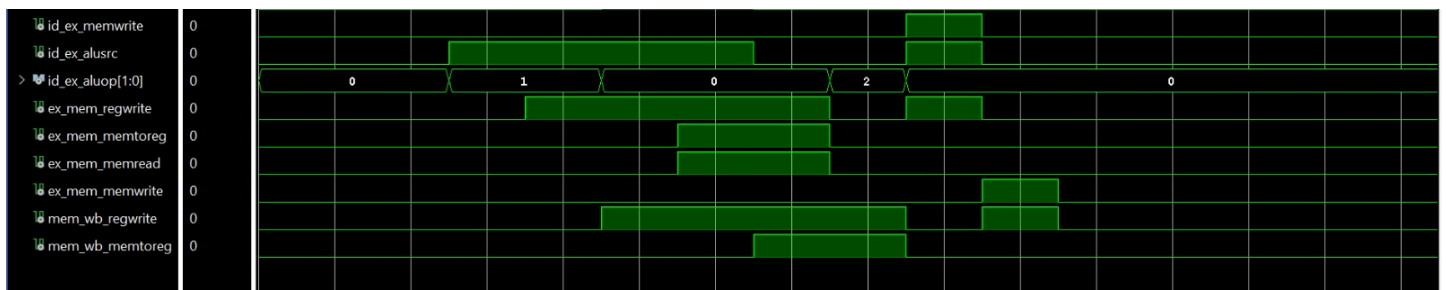
ID/EX Register : 147 bits

EX/MEM Register : 72 bits

MEM/WB Register : 70 bits

Simulation Waveforms





Conclusion

The simulation waveform shows that the 5-stage pipelined processor runs correctly with hazard detection and forwarding. The instruction sequence behaves as expected. The SUBI instruction calculates $\text{reg1} = \text{reg2} - 15 = 0x25 - 0xF = 0x16$. The two LW instructions load values from data memory, resulting in $\text{reg4} = 0xAABBCCDD$ and $\text{reg5} = 0x11223344$. The NAND instruction calculates $\text{reg6} = \sim(\text{reg8} \& \text{reg5})$. With the initialized value $\text{reg8} = 0x0009878C$, we find $\text{reg8} \& \text{reg5} = 0x00000004$, so $\text{reg6} = \sim 0x00000004 = 0xFFFFFFFb$, which matches the waveform. The waveform also shows forwarding activity through the forwardA and forwardB signals, preventing stalls for most dependencies. A single-cycle stall happens between the LW and NAND instructions due to a load-use hazard, confirming that hazard detection is working. After execution finishes, undefined (X) values show up in the pipeline registers because no new valid instructions are fetched, which is expected. The waveform also shows control signals (such as id_regwrite, id_memread, id_memwrite, id_memtoreg, id_alusrc, and id_aluop ... etc), along with their propagation through pipeline registers. These signals correctly correspond to each instruction in every cycle.